

IA PARA DEVS

IAs GENERATIVAS

AULA 04

SUMÁRIO

O QUE VEM POR AÍ?	3
HANDS ON	4
SAIBA MAIS.....	5
O QUE VOCÊ VIU NESTA AULA?	10

EMSE

O QUE VEM POR AÍ?

Chegou o momento de colocar em prática todo o aprendizado que tivemos nas aulas anteriores e realizar um projeto prático para você entender como podemos desenvolver um IA generativa do zero. Vamos lá?

EMANO

HANDS ON

Nesta aula, vamos conhecer como os modelos **“Transformers geram conteúdos”** e as **“limitações das RNN tradicionais”**. Também criaremos um projeto do zero utilizando a **IA generativa da Google com o Gemini**. Nosso objetivo será criar um cardápio fitness utilizando a união de um modelo generativo do gemini com um documento existente treinado por embeddings. Essa técnica é denominada **“RAG” (Retrieval Augmented Generation)**.

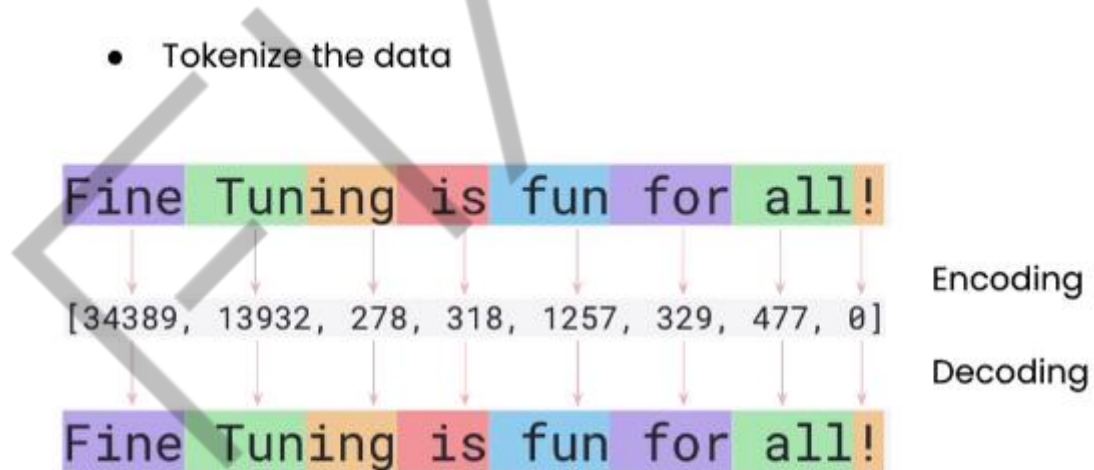
EMANDA

SAIBA MAIS

Entendendo o modelo transformers na prática

Como você já aprendeu sobre o modelo transformers nas aulas anteriores, chegou o momento de colocarmos em prática todo o conhecimento. Recapitulando sobre o assunto, um modelo de Transformer é uma arquitetura de rede neural projetada para tarefas de processamento de linguagem natural (NLP) e outras tarefas sequenciais, como tradução automática, resumo de texto, geração de texto e muito mais. Ele se destaca por sua **capacidade de capturar relações de longo alcance em sequências de entrada sem depender de estruturas sequenciais fixas**, como as encontradas em modelos recorrentes.

Uma das etapas do modelo de transformers é utilizar os **tokens** para criar uma **representação semântica vetorial**, que são os chamados **embeddings**. Mas, como esse processo acontece na prática?



There are multiple popular tokenizers:

- Use the tokenizer associated with your model!

Figura 1: Tokens

Fonte: <https://teetracker.medium.com/llm-fine-tuning-step-tokenizing-caebb280cfc2> (2023)

Basicamente, os tokens são uma representação numérica das palavras para mapear palavras para vetores de números que são utilizados nos embeddings. O modelo mapeia cada token para um vetor de números. O tamanho do vetor (a dimensionalidade) define a quantidade de informações que o embedding captura sobre a palavra. Por meio do treinamento, o modelo aprende as relações semânticas entre os tokens. Isso significa que palavras com significados semelhantes terão embeddings semelhantes no espaço vetorial. Vamos analisar como os tokens são disponibilizados no python?

```
texto = 'Eu vou extrair os insights no banco de dados.'

caracteres = sorted(list(set(texto)))

print(caracteres)

print('Tamanho do seu vocabulário >> ', len(caracteres))
```

Na prática, as palavras são mapeadas por uma representação numérica (como se fosse um índice) e podem ser acessadas de acordo com o número de representação vetorial codificada. Veja, abaixo, um exemplo simples, improvisado apenas para a compreensão do que acontece por trás da “mágica”:

```
letraParaIndex = { lt:i for i,lt in enumerate(caracteres) }
IndexParaLetra = { i:lt for i,lt in enumerate(caracteres) }

encode = lambda s: [letraParaIndex[c] for c in s] # encoder
decode = lambda l: ''.join([IndexParaLetra[i] for i in l]) # decoder

print(encode("Eu vou extrair os insights no banco de dados.))
print(decode(encode("Eu vou extrair os insights no banco de dados.)))
```

Para você conhecer sobre as formas de tokenizar as palavras, é recomendado conhecer a biblioteca [tiktoken](#), que gera tokens de forma fácil e mais eficiente do que o token improvisado no exemplo anterior. Veja um exemplo abaixo:

```
!pip install tiktoken

import tiktoken

enc = tiktoken.get_encoding("gpt2")

enc.encode('Eu vou extrair os insights no banco de dados.')
```

```
#Nosso encoder improvisado

print(encode('Eu vou extrair os insights no banco de dados.'))
```

Agora que você entendeu na prática como funcionam os tokens, vamos criar um projeto de RAG do zero?

Criando um RAG (Retrieve Augmented Generation) com o modelo Gemini

A **Geração Aumentada por Recuperação (RAG)**, em resumo, é uma técnica que **combina o poder de grandes modelos de linguagem (LLMs)** com a **precisão de informações de fontes de dados confiáveis** para fornecer respostas melhores e mais personalizadas.

O LLM recebe uma consulta ou pergunta do usuário e um componente de **recuperação de informações** vasculha um banco de dados relevante em busca de documentos que contenham informações relacionadas à consulta (essas informações podem ser arquivos da sua empresa, informações que podem ser regra de negócio por exemplo). Os documentos recuperados são **concatenados** em um **prompt informativo** e esse prompt fornece ao LLM **contexto adicional** e **instruções específicas** sobre o que o usuário está buscando. Por fim, o LLM já alimentado pelo prompt enriquecido utiliza seus conhecimentos e habilidades para **gerar uma resposta completa e informativa**. A técnica permite direcionar o LLM para **domínios específicos** ou para a **base de conhecimento interna** de uma organização, fornecendo respostas mais adequadas às necessidades do usuário. Basicamente,

estamos aqui unindo a criatividade de uma IA generativa com conhecimentos consolidados de um documento. Incrível, não é mesmo?

E por que vamos trabalhar com a API da Google? Basicamente porque ela está liberada para ser utilizada de forma gratuita por tempo ilimitado. Cada API possui um limite de cota mensal para solicitações e planos gratuitos podem oferecer recursos limitados. Bem, mesmo assim já dá para criarmos projetos bem legais!

Vamos criar um modelo de LLM com RAG? ✎

Primeiro passo, é criar sua API key no ambiente da google. [Aqui](#), você pode criar sua chave de API no Google Studio.



Figura 2 - Criando a API Key
Fonte: elaborado pela autora (2024)

Clique no botão que aparece na tela chamado “Criar chave de API” e crie a sua própria chave de conexão. Depois de criar a chave, precisamos configurar essa chave no ambiente do Google Colab (nas videoaulas estamos ensinando o passo a passo certinho, então não deixe de conferir!).

Gerador de embeddings 📊

Vamos utilizar o modelo **models/embedding-001** que pode ser encontrado mais detalhes [aqui](#).

O desafio

Vamos carregar um conteúdo PDF para alimentar nossa LMM com RAG e gerar o conteúdo a partir dessa base de conhecimento. O conteúdo que vai alimentar a rede neural generativa será um **cardápio fitness**, e vamos criar um modelo que seja capaz

de **gerar uma nova sugestão com base no conteúdo aprendido e de forma mais humanizada**. Bora para o desafio? Não deixe de assistir as videoaulas para aprender todo o passo a passo. **O código estará disponível no github da disciplina!**

EMSE

O QUE VOCÊ VIU NESTA AULA?

Nesta aula, você aprendeu a criar um projeto de IA generativa do zero. Que tal você treinar novas possibilidades e usando novos temas? Não deixe de nos mostrar na comunidade do Discord! Estamos curiosos!

EMANDA

REFERÊNCIAS

DASH, S. Whats is Retrieval-Augmented Generation (RAG)? **Analythics Vidhya**. 2024. Disponível em: <https://www.analyticsvidhya.com/blog/2023/09/retrieval-augmented-generation-rag-in-ai/>. Acesso em: 25 jun. 2024.

GOOGLE IA for developers. Pesquisa de documentos com embeddings. [s.d]. Disponível em: https://ai.google.dev/gemini-api/tutorials/document_search?hl=pt-br. Acesso em: 25 jun. 2024.

TRACKER, T. LLM fine-tuning step: Tokenizing. 2023. Disponível em: <https://teetracker.medium.com/llm-fine-tuning-step-tokenizing-caebb280cfc2>. Acesso em: 25 jun. 2024.

PALAVRAS-CHAVE

IA generativa. LLM. Gemini. Embeddings. RAG.

EMEND



POSTECH